

Chapter 6 In-Class Exercise: Defining and Calling Functions

Prof. Jordan C. Hanson

1 Defining a Function

1. A function is a named block of code that performs a task. A simple Python function is defined using `def`:

```
def square(x):  
    y = x * x  
    return y
```

Here, `x` is a **parameter**. The `return` statement sends the calculated value back to the program.

Call the function using

```
print(square(3))  
print(square(5))
```

Before running the code, predict both results.

2 Arguments and Return Values

1. A value supplied when a function is called is an **argument**. Try

```
a = square(4)  
b = square(6)
```

```
print(a)  
print(b)  
print(a + b)
```

Answer the following:

- What argument is supplied in `square(4)`?
- What value is returned?
- What values are stored in `a` and `b`?

3 A Function with a Pole

1. Consider the mathematical function

$$f(x) = \frac{1}{x-1}.$$

This function has a **pole** at $x = 1$, where the denominator becomes zero.

Define the function in Python:

```
def f(x):  
    return 1 / (x - 1)
```

Evaluate it at several points between 0 and 2:

```
print(f(0.0))  
print(f(0.5))  
print(f(0.9))
```

```
print(f(1.1))  
print(f(1.5))  
print(f(2.0))
```

- What happens to $f(x)$ as x approaches 1 from the left?
- What happens as x approaches 1 from the right?
- Predict what will happen if you execute

```
print(f(1.0))
```

Why?

4 In-Class Exercise

1. Define a new function

$$g(x) = \frac{x+2}{x-1}.$$

Your function should have the form

```
def g(x):  
    # calculate the function  
    # return the result
```

- Evaluate `g(x)` at

$$x = 0, 0.5, 0.9, 1.1, 1.5, 2.$$

Use a simple `for` loop:

```
values = [0, 0.5, 0.9,  
          1.1, 1.5, 2]
```

```
for x in values:  
    print(x, g(x))
```

- Which statement **defines** the function?
- Which expression **calls** the function?
- What value is the **argument** during the first pass through the loop?
- What is returned by `g(2)`?
- Where is the pole of $g(x)$? Explain how you can identify it directly from the denominator.